Distributed systems reliability. Failure tested. Recovery verified.

A real local commerce workload, controlled chaos experiments and evidence-based recovery. Go services own durable business state; Toxiproxy and k6 exercise dependencies; Prometheus, Jaeger and PostgreSQL tell us whether restoration was complete.

REPRODUCIBLE LOCAL STACK

Six Go services | Docker Compose | Six fault presets | 03 October 2026 checkpoint

01 / TRANSACTION ARCHITECTURE

Gateway	Ordering	Inventory	Payments	Worker
REST / SSE	Saga / outbox	Stock reservations	Local provider	Delivery projection

Control-plane isolation

A separate Go control API orchestrates fixed chaos jobs. A private Python runner alone mounts the Docker socket, resolves exact Compose service labels and accepts no arbitrary shell or target. One PostgreSQL instance holds schema-owned business and control records.

02 / RESILIENCE CONTRACTS

Durable saga + compensation

Repeated order keys reuse one logical ID; changed payloads return HTTP 409. Failed payments release reservations and restore stock.

Outbox + messaging

Business changes and outbox events commit together; RabbitMQ confirms durable messages and the worker performs idempotent deliveries.

Reconciliation

Ambiguous payment and incomplete reservation outcomes survive restart, then receive bounded reconciliation.

Bounded dependency attempts

At most 16 concurrent calls, three attempts, an 800 ms default timeout per attempt and small exponential backoff with jitter.

Circuit breakers

Three failed logical calls open the dependency circuit for five seconds; an isolated half-open probe tests recovery. Expected 4xx do not count.

Source-of-truth checks

Completion rejects pending outbox, missing worker deliveries, invalid payment/reservation relationships, active orphan reservations or negative stock.

EVIDENCE RULE A responsive API is not a completed order. Every chaos run checks persisted business invariants after cleaning up the fault.

What happened under failure?

Recorded final inventory experiment (03 October 2026): real k6 load, controlled inventory-proxy injection, Prometheus measurements and post-recovery consistency verification. Not an invented simulation chart.

03 / BASELINE VS FAULT VS RECOVERY

BASELINE	INJECTED FAULT	RECOVERY
0.000	**0.667**	**0.000**
5xx fraction	5xx fraction	5xx fraction
p95 0.048s | 2.000 req/s	p95 0.275s | 1.999 req/s	p95 0.040s | 2.000 req/s

04 / MEASURED CONSISTENCY AND RESTORE

534	0	1.318 s	0.000 s
orders checked	business violations	restore-drill RTO	order-watermark RPO

The restore checks the SHA-256 checksum, exported snapshot counts and persisted order watermark in a separate PostgreSQL recovery database. This RTO covers restore + verification, not service-outage recovery. RPO is the documented order-creation watermark, not a complete broker/state snapshot.

05 / EXPERIMENTS AND TELEMETRY

Six bounded fault types

Inventory unavailable; payment latency; Redis outage; RabbitMQ interruption; worker restart; PostgreSQL network interruption. All six completed in the earlier suite; inventory was rerun in the final checkpoint.

Observed, not invented

Prometheus for request/error/latency, RabbitMQ backlog and circuit signals; OpenTelemetry traces to Jaeger. Ten linked spans across five transaction services were retrieved in the final checkpoint.

06 / VERIFIED BOUNDARIES

Compose health, Go/Vite builds, idempotency, compensation, an injected inventory failure, post-recovery invariants and isolated restore were verified. Kubernetes kind resources were generated but not runtime-executed. The updated CI journey had not run on GitHub at the checkpoint. Nonempty DLQ recovery, coordinated RabbitMQ/PostgreSQL restoration, production HA, internet authentication and real cloud failover were not demonstrated.

SOURCE

github.com/keenosmith-del/sre-chaos-engineering